COP4534 – Project 3

Objective

Students will be able to design, implement, and test solutions for applications of known combinatorial optimization problems. Students will use concepts and tools studied in class and address graphic visualizations of the problem solutions.

Assignment Problem

Design, implement, and test a solution for the following problem. Given a set of n points

$$(x_0, y_0),\ (x_1, y_1),\ \ldots,\ (x_{n-1}, y_{n-1})$$

in the plane representing the coordinates of cities in a map, compute the shortest path that joins all of the points and starts and ends in the same point. This is a application of the traveling salesman problem, in which the graph vertices are represented by the points in the plane and the weights on the edges are represented by the pairwise distances.

Requirements

- The graph will be represented with an adjacency matrix, with distances as weights
- Weights will be positive integer numbers
- The information of the data points will be provided in a text file.
- Number of points will be, at least, 30.
- A *local search* algorithm will be used to provide a solution to the problem
- Output, i.e., points and shortest route, will have a graphical nature.

Please consult the class materials and the textbook, Section 12.6.2, for a discussion on local search.

Input and Output

The input file will have this format:

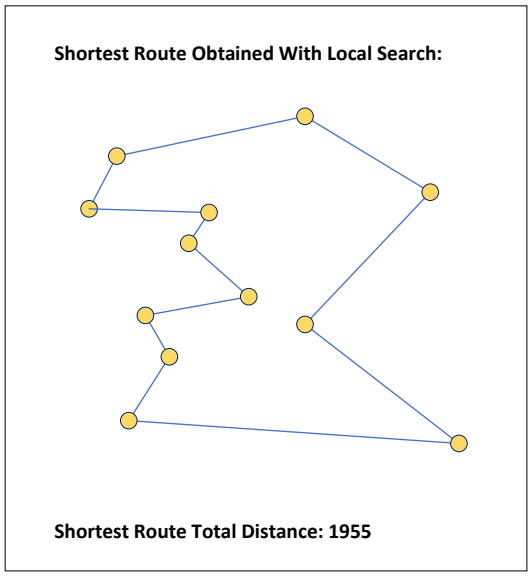
- number of points
- list of the points, one point per line (coordinates will be integers in [50, 600] x [70, 470])

For example,

```
6          ← data set has 6 points
100 82
120 75
88 70      (x, y) points
90 220
345 420
```

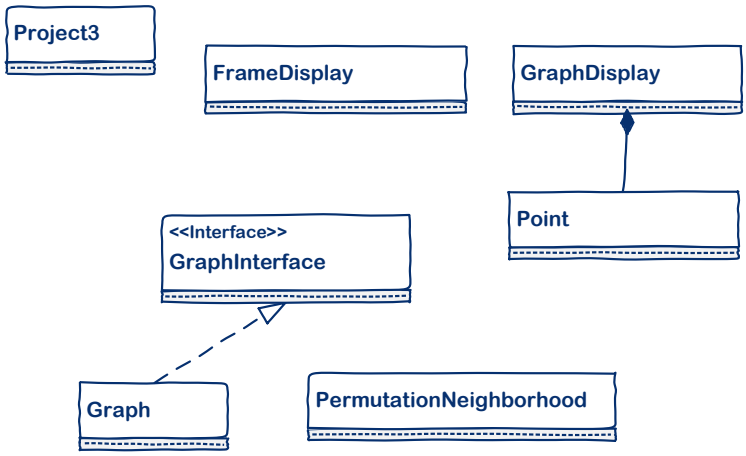
100 317

The output of your program will plot the input points, the shortest route, and will display the distance of the shortest route (in the window with the route and the points, not the console). See below an illustration:



(this is the minimum asked, but feel free to add other information, for example, the length of each edge or labels in the vertices)

UML Class Diagram



Guidelines

- The assignment is to be completed individually or in teams of two/three students.
- The given problem is based on the content studied in class on combinatorial optimization algorithms, Section 19, and your code will be based on the programs discussed in that section.
- Students are required to structure the code as indicated in the UML class diagram above.

- Note that no installation of any software tool, package, or code should be required by anyone interested in running your code, other than a Java IDE and the Java™ Platform, Standard Edition Development Kit.
- Classes from the Java Collection Framework (e.g. java.util.LinkedList, java.util. ArrayList, java.util.PriorityQueue, etc.) are not allowed for use in your implementation of the Graph class. If you have any questions on whether a Java class or utility can be used in your program, don't hesitate to ask.

Deliverables:

- A compressed folder, *PID Project 3* (e.g. *1234567 Project 3*), containing

  1. All of the source code of the exercise
  2. Conclusions (Word or PDF file):
     a) What challenges did you find in implementing a solution to this project?
     b) What did you learn?
     c) Graphical output of the two sets of points you ran the code with (see item **3** below)
  3. Two screenshots of the running program (screenshots are *images*; they include, at least, part of the IDE and the code, and the complete graphical output). The output will correspond to the solution of **two** different point sets, **point_set_30_01.txt** and **point_set_30_02.txt**, which have been provided.

- Students are required to use the given class frameworks included, completing all the methods as indicated in the Javadoc comment section of each method.
- Include **only** the .java files mentioned above; do not include other files or folders generated by the IDE.
- Make sure you write your name(s), section, and Panther ID in the class comment section of each Java file.
- **Team submission: each team member must enter names of all team members in the comment window of the project dropbox. Select a team leader to submit the project (only team leaders submit the project, but the names of the team members must be entered by each student in the team).**

Grading Rubric

The assignment is worth 150 points (out of 1000 total course points). Grade components:

<table>
<tr><th>Component</th><th>Points</th><th>Description</th></tr>
<tr><td>Submission</td><td>5</td><td>The student has submitted the project solution using the requirements for deliverables specified in the Deliverables section.</td></tr>
<tr><td>Organization</td><td>5</td><td>Code is expected to be neat, organized, and readable.</td></tr>
<tr><td>Content</td><td>140</td><td>
<table>
<tr><th>Deliverable</th><th>Points</th></tr>
<tr><td>source code</td><td>100 pts</td></tr>
<tr><td>conclusions</td><td>30 pts</td></tr>
<tr><td>screenshots</td><td>10 pts</td></tr>
</table>
</td></tr>
</table>